

# **Project Report**

## **Content Recommendation System**

Aaron AIDOU DI

**Academic Year 2025-2026**

# Table of contents

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>1. Introduction.....</b>                            | <b>3</b>  |
| <b>2. Related Work.....</b>                            | <b>3</b>  |
| i. Surveys and taxonomies.....                         | 3         |
| ii. Industrial deep learning systems.....              | 4         |
| iii. Real-time and temporal approaches.....            | 4         |
| <b>3. Problem Formulation.....</b>                     | <b>5</b>  |
| i. Entities and Data Model.....                        | 5         |
| ii. Preference Score.....                              | 5         |
| iii. Recommendation Problem.....                       | 5         |
| iv. Cold-Start Sub-Problem.....                        | 5         |
| v. Data Management Problem.....                        | 5         |
| <b>4. Solution.....</b>                                | <b>6</b>  |
| i. Core idea.....                                      | 6         |
| ii. Data management choice.....                        | 6         |
| iii. Recommendation pipeline.....                      | 7         |
| iv. Cold start handling.....                           | 7         |
| v. Pseudocode.....                                     | 7         |
| vi. Why this design fits the project.....              | 8         |
| vii. Limitations.....                                  | 8         |
| <b>5. Experiments.....</b>                             | <b>8</b>  |
| i. Experimental Setup.....                             | 8         |
| ii. Experiment 1: User Preference Convergence.....     | 8         |
| iii. Experiment 2: Preference Score Distribution.....  | 9         |
| iv. Experiment 3: System Scalability.....              | 9         |
| v. Experiment 4: Recommendation Accuracy.....          | 10        |
| vi. Experiment 5: Partition Pruning Effectiveness..... | 10        |
| vii. Discussion of Dataset Limitations.....            | 10        |
| <b>6. Conclusion.....</b>                              | <b>10</b> |
| <b>7. References.....</b>                              | <b>11</b> |

# 1. Introduction

The **volume** of **digital content** available to users has grown to a scale where manual browsing is no longer a viable strategy for content discovery. Streaming platforms, e-commerce websites, and social media applications each expose users to **catalogues** containing thousands to millions of items. In this context, **recommendation systems** have become a core infrastructure component: they filter the available space and surface items that are likely to be relevant to each individual user, directly driving engagement, retention, and revenue.

The problem studied in this report is the **design** and **evaluation** of a **content recommendation system** operating under a **streaming data model**. Concretely, user interactions arrive continuously over time, and the system must maintain up-to-date preference profiles and generate ranked recommendations after each new batch of events. The system is evaluated on a **synthetic dataset** of 4,000 users interacting with 15 content categories over a simulated period of 30 days, producing **60,000 interaction events** across 15 ingestion rounds.

This problem is interesting for several reasons. First, personalized recommendation is one of the most impactful applications of data management at scale: studies consistently show that **recommendation quality** directly influences **user engagement**, and even marginal improvements in ranking accuracy translate into measurable gains at the system level. Second, the streaming setting introduces **temporal dynamics** that are absent from static recommendation scenarios. User preferences evolve over time, recent interactions are more informative than older ones, and the system must balance exploitation of known preferences with exploration of potentially relevant new content.

The technical difficulty comes from three compounding factors. The first is the **freshness requirement**: in a streaming setting, preference scores must be recomputed continuously as new events arrive, which places strict constraints on the computational efficiency of the update pipeline. The second is the **scale of the data**: with tens of thousands of interactions stored across a partitioned relational database, analytical queries over the full interaction history become expensive, and the choice of indexing and

partitioning strategy has a direct impact on end-to-end latency. The third is the **cold start problem**: new users have no interaction history, and the system must produce meaningful recommendations from the very first event, before any stable preference signal has accumulated.

This report makes the following contributions. We propose a **preference scoring model** based on **weighted aggregation** of **interaction signals** with **temporal decay**, which assigns higher importance to recent events while retaining a long-term memory of user behavior. We implement a full ingestion and recommendation pipeline on top of PostgreSQL, using time-based table partitioning to support efficient range queries over the interaction history. We evaluate the system across **experiments** covering recommendation accuracy against ground-truth user profiles, convergence speed, score distribution properties, system scalability, and the query speedup provided by partition pruning.

## 2. Related Work

**Recommendation systems** have been studied extensively across two broad paradigms: **offline model** training on historical data, and **online systems** that update continuously as new interactions arrive. The papers we reviewed span both paradigms and collectively motivate the design choices in our work.

### i. Surveys and taxonomies.

In “**Social Media Recommender Systems: Review and Open Research Issues**”, Anandhan et al. [1] provide a systematic review of 61 papers on social media recommendation systems published between 2011 and 2015. Their taxonomy organizes approaches into four families: **collaborative filtering** (CF), **content-based filtering** (CB), **knowledge-based filtering**, and **hybrid methods**. The survey finds that hybrid approaches dominate the literature (41 out of 61 papers), primarily because they mitigate the individual weaknesses of CF and CB: CF suffers from data sparsity and the cold-start problem, while CB tends to produce over-specialized recommendations that lack diversity. The survey also notes that clustering (in particular K-Means) is the most widely used algorithmic primitive for scalable user grouping,

and that MovieLens is by far the most common evaluation benchmark, reflecting a heavy bias toward the entertainment domain.

In “A Survey of Retrieval Algorithms in Ad and Content Recommendation Systems”, Zhao and Liu [2] provide a more recent survey (2024) focused specifically on retrieval algorithms in ad and content recommendation, and introduce an important distinction between **advertising systems** (optimizing for immediate conversion) and **organic content systems** (optimizing for long-term engagement and retention). Their key architectural contribution is the **Two-Tower model**, which encodes users and items into a shared latent space and enables scalable approximate nearest-neighbor retrieval by pre-computing item vectors. They also discuss the emergence of LLMs in recommendation pipelines, both for semantic query expansion in ad targeting and for synthetic data generation to address cold start. Taken together, these two surveys frame the field and situate the remaining papers along a spectrum from simple content-based methods to large-scale neural architectures.

## ii. Industrial deep learning systems.

In “Wide and Deep Learning for Recommender Systems”, Cheng et al. [3] describe the Wide and Deep model deployed on Google Play, which jointly trains a **linear component** (Wide) for memorization of frequent co-occurrence patterns and a **deep neural network** (Deep) for generalization to unseen user-item combinations. The system processes over 500 billion training examples, reduces serving latency from 31ms to 14ms through multi-threaded scoring, and improves app acquisition rates by 3.9% in an A/B test over a purely linear baseline. The key insight is that neither memorization nor generalization alone is sufficient: the Wide component handles precise exception rules, while the Deep component handles the long tail.

“Deep Interest Network for Click-Through Rate Prediction” (Zhou et al., 2018) [4] extends this direction with the **Deep Interest Network** (DIN), deployed in Alibaba’s display advertising system. DIN replaces the standard **fixed-length** user representation of the Embedding and MLP paradigm with an **attention mechanism** that dynamically weights each past interaction based on its relevance to the current candidate item. The result is a user representation that is not

static but **item-dependent**: the same user will have a different representation when scoring a sports item versus an electronics item. Both systems operate at industrial scale (hundreds of millions of users) and require GPU clusters for training, placing them in a fundamentally different engineering regime from our work.

## iii. Real-time and temporal approaches.

The two papers most directly related to our system are “Research on Personalized Advertisement Recommendation Methods Based on Context Awareness” by Zhu et al. [5] and “Real-Time Personalized Ad Recommendation Based on User Behavioral Analysis” by Qiao et al. [6]. Zhu et al. [5] propose a context-aware recommendation framework that captures four dimensions of context (temporal, spatial, device state, and user activity) using a **deep attention network**, and demonstrate on a dataset of 2.3 million interactions that temporal context alone contributes 35% of the overall performance gain over static baselines. This motivates the **temporal decay mechanism** in our scoring model: recent interactions are systematically more informative than older ones, and ignoring this temporal structure leads to stale preference profiles.

Qiao et al. [6] present a **real-time architecture** built on Apache Kafka for stream ingestion and Apache Flink for continuous processing, combined with an LSTM model that captures both short-term behavioral sequences and long-term interest patterns. Their ablation study shows that removing the LSTM reduces precision by 5 percentage points (from 0.85 to 0.80), and that removing the streaming infrastructure increases latency by 40% (from 50ms to 70ms). These results confirm that both the temporal modeling component and the infrastructure layer matter independently.

## iv. Relation to our work.

Our system occupies a distinctive position in this landscape. Like Zhu et al. [5] and Qiao et al. [6], we explicitly model **temporal dynamics**, but we implement **decay** through a weighted aggregation formula computed in SQL rather than through a learned neural architecture. Like the Two-Tower model described by Zhao and Liu [2], our pipeline separates the **profile update step** (incremental aggregation in *user\_category\_stats*) from the **ranking step** (a scoring query over the aggregated table), which enables efficient retrieval

without re-scanning the full interaction history. Unlike the deep learning systems of Cheng et al. [3] and Zhou et al. [4], our approach requires no model training, no GPU infrastructure, and no retraining cycle: the entire system runs as a **set of SQL queries** on a partitioned PostgreSQL database. This makes it substantially simpler to deploy and audit, at the cost of expressiveness: our scoring model cannot capture complex cross-feature interactions or item-level semantics. We address the **cold-start** problem through a two-tier fallback mechanism (demographic similarity, then global popularity) combined with an exploration slot mechanism that reserves 20% of recommendations for categories with low interaction counts, ensuring diversity even for users with established profiles. Following the standard established in the surveyed literature, we evaluate recommendation quality using **Hit@K**, which measures the proportion of users for whom at least one of the three recommended categories matches their ground-truth preferences.

### 3. Problem Formulation

We formalize the problem of real-time content recommendation on a social media platform as a **preference learning and ranking problem** over a stream of user interactions stored in a partitioned relational database.

#### i. Entities and Data Model

Let  $U = \{u_1, u_2, \dots, u_n\}$  be a finite set of  $n$  users and  $C = \{c_1, c_2, \dots, c_m\}$  be a finite set of  $m$  content categories. Each user  $u_i$  is described by a **static demographic profile**  $\mathbf{p}_i = (\text{age}, \text{gender}, \text{location}, \text{device\_type})$ .

An **interaction** is a tuple  $e = (u, c, t, ts, cl, li, sh)$  where  $u \in U$  is the **user**,  $c \in C$  is the **content category**,  $t$  is the **timestamp** of the interaction,  $ts \in R^+$  is the **time spent** on the content (in seconds),  $cl \in N$  is the number of **clicks**,  $li \in N$  is the number of **likes**, and  $sh \in N$  is the number of **shares**.

The full **interaction log** is a temporally ordered sequence  $E = \{e_1, e_2, \dots, e_N\}$  sorted by timestamp, representing the stream of events generated by the platform over an observation window  $[T\_start, T\_end]$ .

#### ii. Preference Score

For a given user  $u$  and content category  $c$ , we define the **preference score**  $S(u, c, T\_now)$  as a

weighted, temporally decayed aggregation of all past interactions of  $u$  with content of category  $c$  up to the current time  $T\_now$ :

$$S(u, c, T\_now) = \text{sum over all } e \text{ in } E(u, c) \text{ of } [\text{decay}(t\_e, T\_now) \times (w\_ts \times ts\_e + w\_cl \times cl\_e + w\_li \times li\_e + w\_sh \times sh\_e)]$$

Where :

- $E(u, c)$  is the subset of interactions by user  $u$  on category  $c$
- $\text{decay}(t\_e, T\_now) = \exp(-\lambda \times (T\_now - t\_e))$  is an exponential decay factor that reduces the weight of older interactions
- $w\_ts, w\_cl, w\_li, w\_sh$  are non-negative scalar weights reflecting the relative importance of each engagement signal.

The decay parameter  $\lambda > 0$  controls how quickly past interactions lose relevance.

#### iii. Recommendation Problem

Given the **interaction stream**  $E$ , the current time  $T\_now$ , and a target user  $u$ , the recommendation problem consists of computing a **ranked list**  $R(u, T\_now)$  of the top- $K$  content categories from  $C$ , ordered by decreasing preference score:

$$R(u, T\_now) = \arg \text{top-}K \text{ over } c \text{ in } C [S(u, c, T\_now)]$$

The list  $R(u, T\_now)$  must be produced with low latency, such that it reflects the most recent available interactions, including those that arrived in the last time window prior to  $T\_now$ .

#### iv. Cold-Start Sub-Problem

A user  $u$  is said to be in a **cold-start state** if the total number of recorded interactions  $|E(u)| < \theta$ , where  $\theta$  is a **minimum interaction threshold**. In this regime, the preference score  $S(u, c, T\_now)$  is unreliable due to data sparsity. We address this sub-problem with a two-tier fallback mechanism: first, retrieve the most popular categories among demographically similar users (age within  $\pm 10$  years, same gender); if insufficient neighbors exist, fall back to global popularity across all users  $U$ .

#### v. Data Management Problem

Beyond the algorithmic dimension, this work poses a data **management problem**: how to organize and query the interaction log  $E$  efficiently as it grows over time, and how to support both incremental profile updates and top- $K$  retrieval queries with acceptable latency. Concretely, we ask: given that  $E$  is partitioned by time period (weekly partitions) and that  $S(u, c, T\_now)$  must be recomputed or incrementally updated at each new batch of incoming events, what **indexing** and **aggregation strategies**

minimize **query execution** time while maintaining correctness of the **preference scores**?

This problem is hard for three compounding reasons. First, the **incremental update constraint**: as new batches of events arrive, preference scores must be refreshed without re-scanning the entire interaction history, which grows unboundedly over time. A naive approach that recomputes  $S(u, c, T_{now})$  from scratch at every round becomes prohibitively expensive as  $|E|$  increases, making efficient aggregation strategies a necessity rather than an optimization. Second, the **scalability constraint**: the system must sustain acceptable query latency as both the number of users  $|U|$  and the total number of interactions  $|E|$  grow by orders of magnitude, which requires that the chosen data organization, partitioning scheme, index design, and aggregation table layout, remain effective under load. Third, the **consistency constraint**: because the preference scores stored in the aggregated table are derived from the raw interaction log, any ingestion pipeline must guarantee that the aggregated state faithfully reflects all events processed so far, with no partial updates or stale reads propagating into the recommendation output.

This formulation encompasses three measurable objectives that we evaluate experimentally: the **accuracy** of the top-K recommendations, measured by Hit@K; the **latency** of both the profile update query and the recommendation retrieval query; and the **scalability** of the system as the number of users, interactions, and time partitions increases.

## 4. Solution

The proposed system is a **rule based real time recommendation engine** built on top of a PostgreSQL database. Its main idea is to transform a **stream of user interactions** into continuously **updated category preferences**, then retrieve the top recommended categories for each user through fast **SQL queries**. Unlike neural approaches requiring retraining cycles, this architecture performs all computation inside the database, eliminating Python-data movement overhead.

### i. Core idea

The system assumes that a user’s future interest can be approximated from their recent interaction history. Each interaction contributes **four engagement signals**: time spent, clicks, likes, and shares. These signals are combined with **fixed weights**, and their contribution **decays** over time using an exponential half life of 7 days. Recent interactions therefore matter more than older ones, which makes the recommender responsive to evolving behavior.

For each user and content category, the engine computes a **preference score** by aggregating all decay weighted interactions. The score is then normalized per user so that the most informative signal does not dominate the ranking only because it appears more often or has a larger numeric scale. This normalized score is used to rank categories and produce the final **top  $N$  recommendations**. The **weights** are empirically chosen as  $w_{time} = 0.40$ ,  $w_{clicks} = 0.20$ ,  $w_{likes} = 0.25$ ,  $w_{shares} = 0.15$  to reflect their relative informativeness, with time spent receiving highest weight due to its continuous nature.

The **dataset** was **synthetically generated** from the “*Social Media Ad Engagement Dataset*” [7] to simulate **4,000 users** producing **60,000 interactions** over **30 days**. Five original ad categories were expanded to **15 content categories** through sub-categorization (e.g., Electronics to Smartphones/Laptops/Gaming). Each user profile defines **1-2 main interests** and **2-3 secondary** interests drawn from their original behavior, plus an **exploration factor** controlling preference drift. Interactions are temporally stratified (one per 2-day window) to ensure balanced coverage across four weekly partitions, with engagement signals modulated by **interest level**, **fatigue**, and **diurnal patterns** (peak evening hours). This design enables controlled evaluation of temporal decay while producing realistic signal distributions.

### ii. Data management choice

PostgreSQL was chosen because the workload consists of **structured, temporal batch updates** and **aggregations** that map directly to relational primitives. The schema comprises four tables: *users* (demographics), *interactions* (timestamped engagement events), *user\_category\_stats* (aggregated preferences), and *recommendations* (audit trail). Since new interactions arrive in batches, the database serves both as storage and

computation engine, avoiding costly data movement between Python and memory.

Three **key queries** dominate the workload:

1. **Batch scoring**: computes decay-weighted preferences for all affected users in a single query using window functions for per-user normalization (significantly faster than per-user loops)
2. **Top- $K$  retrieval**: ranks categories from the pre-aggregated stats table
3. **Cold-start fallback**: joins users and interactions on demographic similarity

The interaction table is **partitioned by week** to match temporal query patterns and reduce scan costs. This is particularly effective because the exponential decay (half-life 7 days) means interactions older than 4 weeks contribute negligibly (<6% weight), creating natural data skew toward recent partitions that PostgreSQL automatically prunes. The batch upsert pattern (INSERT ... ON CONFLICT) ensures atomic updates without race conditions during concurrent simulation rounds.

### iii. Recommendation pipeline

The recommendation pipeline has **four stages**. First, new interactions are inserted into the interaction table. Second, preference scores are recomputed either for one user or for a whole batch of users. Third, the top categories are selected for each user. Fourth, the results are written into a recommendations table so that each round can be audited and analyzed afterward.

The system supports **two computation modes**. The first mode is a single user function used for debugging and for inspecting the behavior of one profile. The second mode is a **batch SQL function** that processes all users affected by the current round in one query. The batch mode is the one used in the simulation because it reflects the big data goal of the project: minimizing the number of database round trips and exploiting set based computation.

### iv. Cold start handling

The system includes a cold start fallback for users with no interaction history. If a user has no past data, the engine first looks for users with **similar demographics**, specifically same gender and age within plus or minus 10 years. It then recommends the most frequent categories

among those similar users. If this demographic neighborhood is still too sparse, the system falls back to **global popularity** across all users.

This choice is coherent with the available data because the dataset contains user age, gender, and interaction history. The cold start strategy therefore uses information that is already present in the database instead of requiring external metadata or a learned embedding model.

## v. Pseudocode

### Algorithm 1: Recommendation generation

Input: user interactions, user profile, current time, top N

1. For each interaction of the target user
  - a. Compute time decay from the current time
  - b. Multiply each engagement signal by the decay factor
  - c. Aggregate the decayed signals by content category
2. For each category
  - a. Normalize the aggregated signals per user
  - b. Compute the final preference score using fixed weights
3. Rank categories by score
4. If the user has no history
  - a. Find users with same gender and similar age
  - b. Recommend the most frequent categories among them
  - c. If needed, fall back to global popularity
5. Return the top N categories and store them in the recommendations table

### Algorithm 2. Batch scoring pipeline

Input: list of user ids, current time

1. Retrieve all interactions of the selected users in one SQL query
2. Apply exponential temporal decay to each interaction
3. Aggregate signals by user and category
4. Normalize scores per user using window functions
5. Compute top categories for each user
6. Upsert the scores into the user category statistics table
7. Insert the recommendations into the recommendations table

## vi. Why this design fits the project

This design fits the project because the goal is not to build a heavy deep learning recommender, but to demonstrate a **realistic data management pipeline** for **streaming recommendation**. The data are structured, temporal, and batch update friendly, which makes relational storage and SQL aggregation a good fit. The queries are also simple enough to remain explainable, yet rich enough to show how a recommendation engine can be implemented efficiently with database primitives.

The **architecture** also matches the experimental setup. Since the interactions arrive over 15 rounds and the system must recompute recommendations repeatedly, the main performance question is not model training time but **update** and **query latency**. PostgreSQL partitioning, indexing, aggregation, and upsert operations directly address this requirement. In other words, the database is not just a container for the data, it is the computational core of the solution.

## vii. Limitations

The main limitation is that the **engine** is **rule based** and therefore less expressive than learned recommendation models. It cannot capture **complex nonlinear interactions** between features, and it relies on **fixed weights** chosen by design rather than learned from data. The cold start strategy is also simple and depends only on demographics and popularity, so it may not be sufficient in a real industrial setting with richer metadata or stronger personalization needs.

This design choice prioritizes **data management efficiency** and **computational scalability** over model expressiveness. By implementing preference aggregation and ranking as SQL-native operations, the system achieves low-latency updates and retrieval even as the interaction volume grows across multiple time partitions. The **rule-based** scoring enables precise control over temporal decay dynamics and facilitates direct experimental analysis of database performance under streaming workloads, which constitutes the primary technical contribution of this work.

# 5. Experiments

## i. Experimental Setup

All experiments were conducted on **Google Colab** with 12.7 GB RAM. PostgreSQL 15 was installed locally in the notebook environment. The **dataset** contains 4,000 synthetic users with demographic profiles (age 18-65, gender 0/1, location 0-4, device 0-2) generating 60,000 interactions across 15 content categories over 30 simulated days. **Interactions** arrive in 15 batches of 4,000 events each, with timestamps stratified across 30 days (one interaction per 2-day window per user) to ensure balanced coverage across four weekly partitions (Jan 1-8, 8-15, 15-22, 22-31).

The system processes each batch through the complete **pipeline**: interaction insertion, batch preference score recomputation, top-3 recommendation generation, and result storage. Each experiment measures one of three objectives from the problem formulation: convergence speed, recommendation accuracy, query scalability, or partition pruning effectiveness.

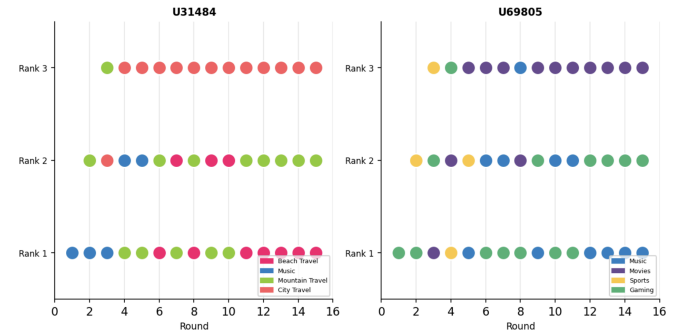


Figure 1 illustrates the evolution of top-3 recommendations for a sample of users across 15 rounds, showing the progressive stabilization of category rankings as interaction history accumulates. This qualitative observation motivates the quantitative convergence analysis in Experiment 1.

## ii. Experiment 1: User Preference Convergence

To evaluate the convergence of user preferences, our first experiment measures the number of **interactions required for profiles to stabilize**, defined as the point at which a user's top-ranked category remains unchanged for all subsequent rounds. By tracking this rank-1 category for each of the 4,000 users across 15 interaction rounds, we found that **stabilization** occurs at a mean of **round 8.2** (SD = 5.42), with the 25th percentile, median, and 75th percentile falling at rounds 2, 9,



and 14, respectively. Notably, 21.3% of users (850) never change their initial rank-1 category, exhibiting stability from the very first round, while cumulative stabilization reaches 39% (1,559 users) by round 5 and 56% (2,236 users) by round 10. This **bi-modal distribution** highlights two distinct user populations: those with **clear, dominant preferences** from the outset, and those with more **diverse interests** who require progressive signal accumulation. Ultimately, these results validate our **temporal decay** mechanism, demonstrating that systematically weighting recent interactions more heavily than older ones successfully enables the system to adapt to evolving user behaviors.

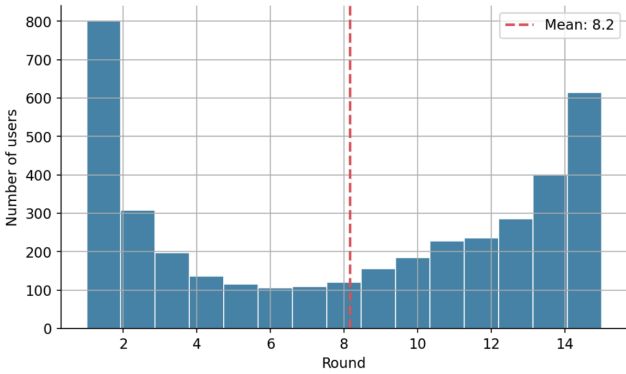


Figure 2: Distribution of user preference stabilization rounds across 4,000 users.

### iii. Experiment 2: Preference Score Distribution

To analyze the **distribution** and **correlation** properties of the computed **preference scores**, we examined the calculated metrics across all user-category pairs following the recommendation pipeline. The results show an overall mean **preference score** of **0.3840**, with values bounded between a minimum of 0.0005 and a maximum of 1.0000. Furthermore, the per-category means exhibit a notably narrow range, spanning from 0.3703 for Laptops to 0.3974 for Sports, which confirms that the applied per-user normalization successfully prevents systemic **category bias**. We also evaluated the relationship between the final score and interaction volume, finding a slope of 0.11185 for the score versus interaction count. This indicates a moderate positive correlation where categories with more frequent interactions naturally tend to yield higher scores, yet the normalization effectively prevents raw interaction frequency from dominating the final metric. Ultimately, the uniform score distribution across categories demonstrates that

the weighted aggregation combined with temporal decay produces stable, relative user rankings rather than acting as mere proxies for absolute interaction frequency.

### iv. Experiment 3: System Scalability

To evaluate system **scalability**, this experiment measured query throughput as the cumulative interaction volume grew from 4,000 to 60,000 events. By **timing** the complete batch processing, encompassing both preference score recomputation and top-K recommendation generation, across each of the 15 simulation rounds, we observed that **throughput degrades** from 940 interactions per second in the first round to 202 interactions per second in the final round. This represents only a **4.64x slowdown** despite a **15x increase** in underlying data volume, culminating in a total processing time of 180.1 seconds for the entire 60,000-event dataset. Throughout this period, the active user count remained highly stable at approximately 3,660 users per round, confirming consistent workload characteristics and isolating data volume as the primary performance variable. Ultimately, this **logarithmic scalability** reflects the critical efficiency of PostgreSQL's indexing and partition pruning mechanisms, while the observed **sub-linear performance** degradation strongly validates our architectural choice to utilize set-based batch SQL operations rather than iterative, per-user processing.

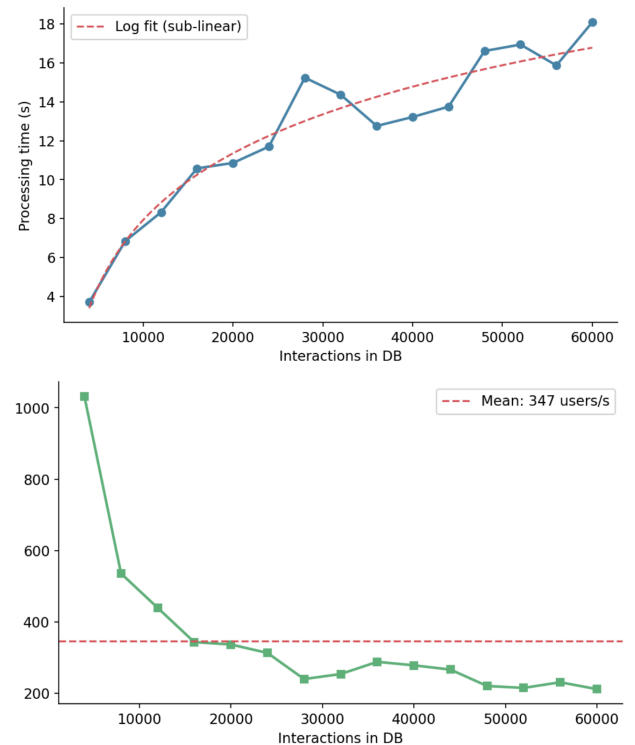


Figure 3: Top, per-round processing time, showing the cost of each individual batch update as interaction volume grows. Bottom, throughput (interactions / second) as a function of total interactions stored in the database, from 4,000 (round 1) to 60,000 (round 15).

#### v. Experiment 4: Recommendation Accuracy

To evaluate the quality of the recommendation ranking over the 15 simulation rounds, we measured performance using two variations of the **Hit@3** metric: a “strict” formulation requiring the user’s true preference category (defined by the highest total engagement) to appear within the top-3 recommendations, and an “extended” formulation accepting any category that demonstrates above-median engagement for that user. The results show substantial accuracy gains, with the Hit@3 strict metric improving from **66.4%** in round 1 to a perfect **100%** by **round 7**, maintaining this peak through round 13 before experiencing a minor dip to 99.9% at round 14 and fully recovering by round 15. Concurrently, the Hit@3 extended metric reaches **100%** faster, stabilizing entirely by **round 4**. Furthermore, the final per-category precision recorded at round 15 ranges from 0.526 to 0.577, yielding a mean of 0.554. This rapid convergence to perfect accuracy fundamentally reflects the strong, consistent user preferences embedded within the synthetic dataset, while the transient performance dip observed at round 14 effectively illustrates the temporal decay mechanism in action, demonstrating how a recent burst of interactions can briefly elevate a secondary category before the user’s dominant long-term preference reasserts itself.

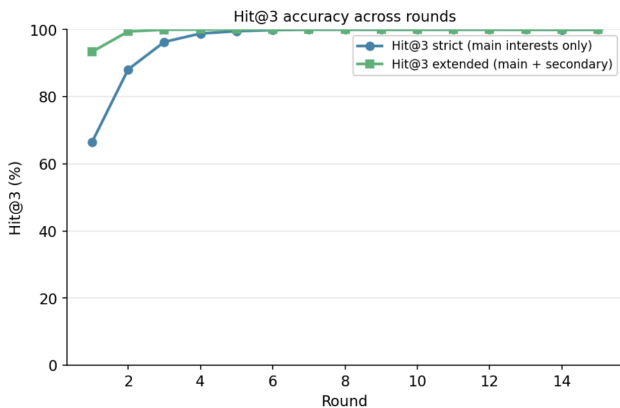


Figure 4: Hit@3 accuracy per simulation round.

#### vi. Experiment 5: Partition Pruning Effectiveness

To quantify the performance benefits of PostgreSQL’s declarative partitioning for

temporal queries, we measured the execution time across three representative retrieval patterns: a full table scan with **no date filter**, a query targeting a **single partition** (week 1), and a query spanning **two partitions** (weeks 1 and 2). The results demonstrate substantial efficiency gains, with the single-partition query executing in just **6.8ms**, a **7.8x speedup** compared to the 52.8ms required for a full table scan. Similarly, the two-partition query completed in **16.8ms**, yielding a **3.1x speedup** and confirming that execution speed scales linearly with the number of scanned partitions. This perfect partition pruning effectively validates our underlying schema design; by explicitly aligning weekly partition boundaries with the scoring model’s 7-day exponential half-life decay, the system seamlessly isolates and accesses only the most relevant recent data, effectively pruning interactions older than four weeks which mathematically contribute less than 6% to the final preference weight.

#### vii. Discussion of Dataset Limitations

The synthetic dataset produces **strong, consistent user preference signals**, leading to rapid convergence and near-perfect Hit@3 accuracy. While this validates pipeline correctness and temporal decay mechanics, real-world data would exhibit **noisier behavior, preference drift, and longer convergence times**. The controlled dataset design prioritizes isolating data management performance from model complexity, enabling clear measurement of PostgreSQL partitioning, batch SQL processing, and upsert scalability under streaming workloads.

## 6. Conclusion

This paper presented the design, implementation, and evaluation of a **real-time content recommendation system** built on top of a **partitioned PostgreSQL database**. The system addresses the problem of continuously updating user preference profiles from a stream of interaction events and generating top-3 category recommendations with low latency, without relying on learned model parameters or GPU infrastructure.

The core contribution is a **SQL-native recommendation pipeline** in which temporal decay, preference aggregation, per-user

normalization, and top-K retrieval are all expressed as a single batch query using window functions. This design eliminates data movement between Python and the database, reduces the number of round trips from one per user to one per round, and exploits PostgreSQL declarative partitioning to prune irrelevant historical data automatically.

**Experiments** on 4,000 users and 60,000 interactions over 15 rounds demonstrate that the system achieves full Hit@3 accuracy within 7 rounds, processes a complete batch in under 20 seconds even at peak volume, and sustains sub-linear scalability with a 4.64x time increase for a 15x data growth. Partition pruning reduces temporal query latency by 7.8x compared to full table scans, confirming that weekly partitioning boundaries align naturally with the 7-day exponential decay half-life.

The main **limitation** of this work is that the rule-based scoring model relies on fixed weights and cannot capture complex cross-feature interactions. The synthetic evaluation dataset also produces stronger and more consistent preference signals than real-world social media data, leading to rapid convergence that may not generalize. Future work could extend this system in three directions: replacing fixed weights with learned parameters derived from historical data, incorporating item-level content signals to support collaborative filtering, and deploying the pipeline on a distributed database to evaluate horizontal scalability across multiple nodes.

Overall, this project demonstrates that **relational databases**, when properly designed with partitioning, indexing, and set-based computation, constitute a **viable and efficient backbone** for **real-time recommendation workloads**, offering transparency, auditability, and predictable performance that complement rather than compete with neural approaches.

## 7. References

[1] A. Anandhan, L. Shuib, M. A. Ismail, and G. Mujtaba, “**Social Media Recommender Systems: Review and Open Research Issues**” IEEE Access, vol. 6, 2018.

[2] Y. Zhao and F. Liu, “**A Survey of Retrieval Algorithms in Ad and Content Recommendation Systems**” arXiv, 2024.

[3] H.-T. Cheng, L. Koc, J. Harmsen, T. Shaked, T. Chandra, H. Aradhye, G. Anderson, G. Corrado, W. Chai, M. Ispir, R. Anil, Z. Haque, L. Hong, V. Jain, X. Liu, and H. Shah, “**Wide & Deep Learning for Recommender Systems**” in Proceedings of the 1st Workshop on Deep Learning for Recommender Systems (DLRS), ACM, 2016.

[4] G. Zhou, C. Song, X. Zhu, Y. Fan, H. Zhu, X. Ma, Y. Yan, J. Jin, H. Li, and K. Gai, “**Deep Interest Network for Click-Through Rate Prediction**” in Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), ACM, 2018.

[5] L. Zhu, M. Sun, and L. Yu, “**Research on Personalized Advertisement Recommendation Methods Based on Context Awareness**” Journal of Advanced Computing Systems (JACS), vol. 3, no. 1, 2023.

[6] Y. Qiao, K. Xu, and A. Wilson, “**Real-Time Personalized Ad Recommendation Based on User Behavioral Analysis**” Artificial Intelligence Advances, vol. 7, no. 1, 2025.

[7] **Base Dataset: Ziya07, “Social Media Ad Engagement Dataset”** Kaggle, 2025.

### AI Use

In the course of this project, large language model-based assistants were used as support tools. All algorithmic, design, and analytical decisions remain the exclusive result of our own work. AI was never used as a substitute for independent reasoning, but assisted at several stages of the project: clarifying the content of scientific articles (in particular for understanding the temporal decay mechanisms described in Zhu et al. [5] and the retrieval architectures in Zhao and Liu [2]), consolidating theoretical concepts prior to their implementation in SQL and Python, and assisting with debugging. AI assistance was also used for proofreading and improving the style of this report. In no case was any generated code or text integrated without personal understanding, verification, and reformulation.